فصل ۵: مدل‌های خطی– رگرسیون logistic

مقدمه

رگرسیون خطی یک روش بسیار قدیمی و بخشی از آمار است. این روش در یادگیری ماشین شامل یک مجموعه آموزش و آزمایش است. به این ترتیب می‌توان با استفاده از cross-validation با مدل‌ها و الگوریتم‌های دیگر، آن را مقایسه کرد. رگرسیون خطی سنتی روی کل مجموعه داده آموزش داده و آزمایش می‌کند. این هنوز هم یک روش معمول است، احتمالاً به این دلیل که کم برازش (under fitting) از بیش برازش (overfitting) بیشتر رخ می‌دهد.

کلاسه‌بندی با رگراسیون logistic

در برخی از زمینه‌ها، قبل از تعیین پیش‌بینی کلاسه‌بندی، رگرسیون احتمال تعداد واقعی یک کلاس را محاسبه می‌کند. با این حساب، بیایید مجموعه داده‌های دیابت Pima Indians را که توسط دانشگاه کالیفرنیا، ایروین (UCI) ارائه شده است، بررسی کنیم.

برای دانلود به لینک زیر مراجعه نمایید

http://networkrepository.com/pima-indians-diabetes.php

حال به کمک ماژول panda داده‌ها را بارگذاری می‌کنیم

```
import pandas as pd
data_web_address ="pima-indians-diabetes.csv" # ادرسی که فایل در سیستم شما ذخیره شده است
column_names =
['pregnancy_x','plasma_con','blood_pressure','skin_mm','insulin','bmi','pedigree_
func','age','target']
feature_names = column_names[:-1]
all_data = pd.read_csv(data_web_address , names=column_names)
all_data.head()
```

جدول (۵-۱): نمایش داده‌ها

	pregnancy_x	plasma_con	blood_pressure	skin_mm	insulin	bmi	pedigree_func	age	target
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

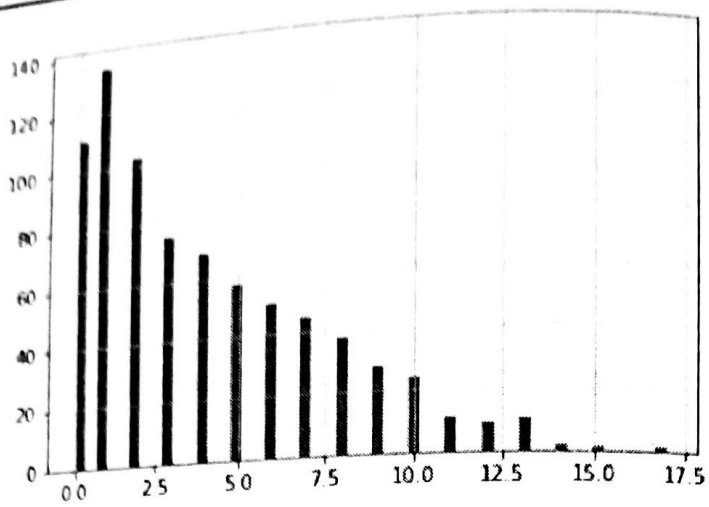
```
all_data.describe()
```

جدول (۵-۲): توضیحات در مورد داده‌ها

	pregnancy_x	plasma_con	blood_pressure	skin_mm	insulin	bmi	pedigree_func	age	target
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

هیستوگرام بارداری _ را ترسیم کنید

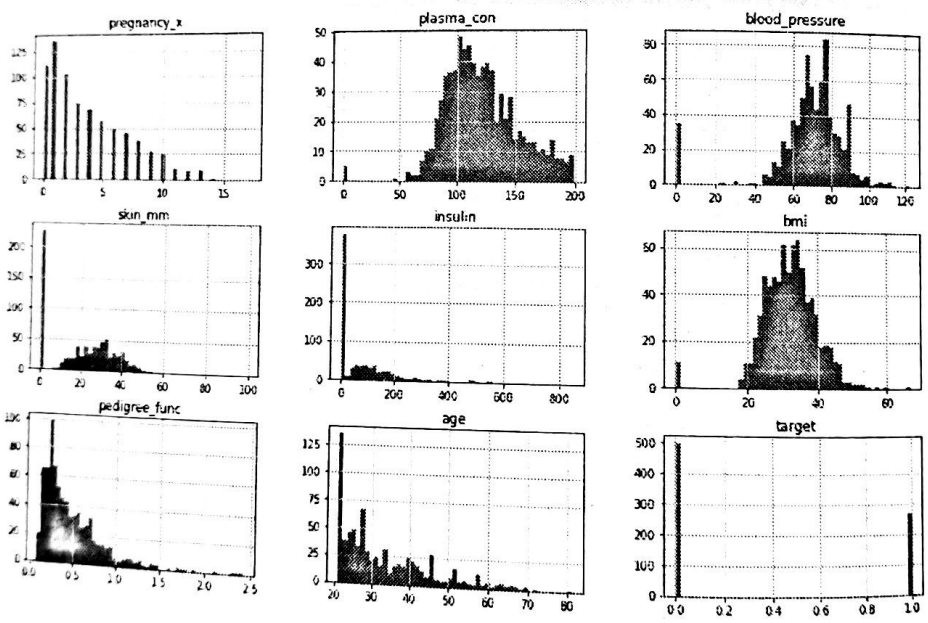
```
import matplotlib.pyplot as plt
plt.hist(all_data.pregnancy_x , bins=50)
plt.grid(True)
plt.show()
```



شکل (۵-۱): نمودار بارداری

نمودار همه ستون‌ها را نمایش می‌دهیم

```
all_data.hist(figsize=(15,9),bins=50)
```



شکل (۵-۲): نمودار ستون‌های مختلف

Blood_pressure و bmi به غیر از صفرهای ناهنجار مانند توزیع‌های طبیعی به نظر می‌رسند. با توجه به نمودار، داده‌های target، توازن ندارند برای مشاهده از دستور زیر استفاده نمایید

```
all_data.target.value_counts()
output: 0  500
1  268
Name: target, dtype: int64
```

یادگیری ماشین با رگرسیون لجستیک (Logistic regression)

شما با مراحل آموزش و تست طبقه‌بندی آشنا هستید. با رگرسیون لجستیک، موارد زیر را انجام خواهیم داد

- داده‌ها را به ترتیب در آرایه‌های ویژگی x و هدف y بارگذاری کنید
- داده‌ها را به مجموعه‌های آموزشی و آزمایشی تقسیم کنید
- طبقه‌بندی رگرسیون لجستیک را روی مجموعه آموزش آموزش دهید
- عملکرد طبقه‌بندی را روی مجموعه تست آزمایش کنید

متغیرهای ورودی X و متغیر هدف y را تنظیم کنید

```
import numpy as np
import pandas as pd
X = all_data[feature_names]
y = all_data['target']
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=7,stratify=y)
```

به stratify=y توجه کنید که متغیر y را طبقه‌بندی می‌کند. این به این معنی است که در هر دو y_test و y_train و نسبت‌های صفر و یکسانی وجود دارد.

رگرسیون لجستیک را آموزش و پیش‌بینی را انجام می‌دهیم

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression()
lr.fit(X_train,y_train)
y_pred = lr.predict(X_test)
```

دقت پیش‌بینی را با accuracy_score، مشاهده می‌کنیم

```
from sklearn.metrics import accuracy_score
accuracy_score(y_test,y_pred)
output: 0.7857142857142857
```

بنابراین، ما امتیازی کسب کرده ایم، اما آیا این امتیاز بهترین معیاری است که می‌توانیم تحت این شرایط استفاده کنیم؟ آیا می‌توانیم بهتر عمل کنیم؟ شاید بله .به ماتریس سردرگمی (confusion matrix) نتایج به شرح زیر نگاه کنید.

بررسی خطاهای رگرسیون لجستیک با یک ماتریس سردرگمی(confusion matrix)

confusionmatrix را برای رگرسیون لجستیکی که ساخته‌ایم وارد کرده و مشاهده می‌کنیم

ماتریس سردرگمی با سه متغیر آرگومان فراخوانی می‌کنم:

- y_test: مجموعه هدف آزمایش
- y_pred: پیش‌بینی‌های رگرسیون لجستیک ما
- labels: برچسب کلاس مثبت

برچسب‌ها = [۱،۰] به این معنی است که کلاس مثبت ۱ و کلاس منفی ۰ است.

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, y_pred,labels = [1,0])
output: array([[33, 21],
   [12, 88]], dtype=int64)
```

خواندن ماتریس سردرگمی

آرایه اعداد (که در بالا به دست آمد) به معنای زیر است:

جدول (۵-۳): مثال ماتریس سردرگمی

		پیش‌بینی	
		پیش‌بینی داشتن دیابت	پیش‌بینی نداشتن دیابت
واقعیت	دیابت دارند	۳۳ نفر به درستی پیش‌بینی شدند که دیابت **دارند**	۲۱ نفر که دیابت دارند، پیش‌بینی شدند که دیابت ندارند
	دیابت ندارند	۱۲ نفر که دیابت ندارند، پیش‌بینی شدند که دیابت دارند	۸۸ نفر به درستی پیش‌بینی شدند که دیابت **ندارند**

ماتریس سردرگمی (confusion matrix)

در حالت کلی این ماتریس به صورت زیر است:

جدول (۵-۴): ماتریس سردرگمی

		پیش‌بینی	
		Positive Class	Negative Class
واقعیت	Positive Class	True Positive (TP)	False Negative (FN)
	Negative Class	False Positive (FP)	True Negative (TN)

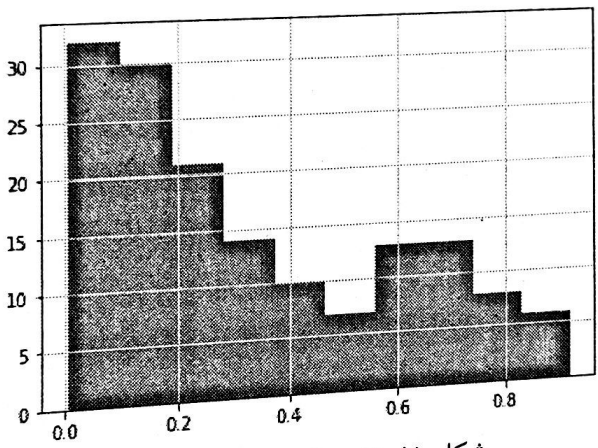
متفاوت بودن آستانه کلاسه‌بندی در رگرسیون لجستیک

ما از این واقعیت استفاده خواهیم کرد که در طبقه‌بندی رگرسیون لجستیک، رگرسیون وجود دارد تا تعداد دفعاتی که افراد به دلیل نداشتن دیابت به خانه فرستاده شده اند ولی دیابت داشته‌اند، به حداقل برسد.

```
y_pred_proba = lr.predict_proba(X_test)
y_pred_proba
output: array([[0.90584106, 0.09415894],
  [0.89447658, 0.10552342],
  [0.87787275, 0.12212725],
  ..........
```

در ردیف اول، یک احتمال در حدود ۰.۹ به کلاس صفر و یک احتمال ۰.۰۹ به ۱ اختصاص داده می‌شود. توجه داشته باشید که به عنوان احتمال، این اعداد تا ۱ جمع می‌شوند. (یعنی جمع ردیف‌ها ۱ است) (شکل ۵-۳).

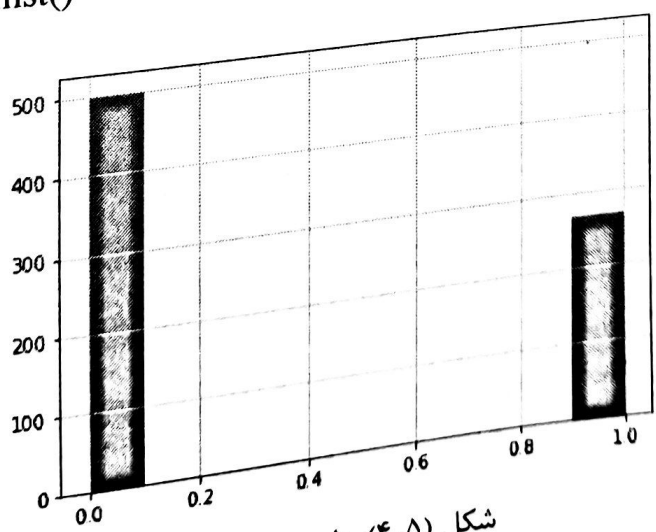
```
pd.Series(y_pred_proba[:,1]).hist()
```



شکل (۵-۳): نمایش پیش‌بینی

در رگرسیون لجستیک، اگر احتمال بیشتر از ۰.۵ یا نصف باشد الگوریتم به طور پیش فرض فقط ۱ را انتخاب می‌کند. اکنون، این را با هیستوگرام هدف در ابتدا مقایسه کنید

```
all_data['target'].hist()
```



شکل (۵-۴): داده‌های هدف

• برای انتخاب کلاس دسته‌بندی براساس یک آستانه، از ماژول binarize استفاده کنید
ابتدا آن را فراخوانی کنید
• به دو ستون اول y_pred_proba نگاه کنید
• سپس توابع binarize را در y_pred_proba با آستانه 0.5 امتحان کنید.

```
from sklearn.preprocessing import binarize
y_pred_proba
output: array([[0.90584106, 0.09415894],
   [0.89447658, 0.10552342],
   [0.87787275, 0.12212725],
.....
y_pred_default = binarize(y_pred_proba, threshold=0.5)
y_pred_default
output: array([[1., 0.],
   [1., 0.],
   [1., 0.],
   [1., 0.],
```

اگر مقدار در y_pred_probaare بزرگ‌تر از 0.5، آرایه را با مقادیر ۱ پر می‌کند؛ در غیر اینصورت یک عدد صفر قرار می‌دهد. در ردیف اول، 0.90 بزرگ‌تر از 0.5 است در حالی که 0.09 اینگونه نیست. بنابراین برای باینری شدن، 0.90 را با 1 و 0.09 را با 0 جایگزین کنید.

اکنون، ستون اول y_pred_default را در نظر بگیرید.

```
y_pred_default[:,1]
output: array([0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 1., 0., 0., 1., 0., 1.,
   0., 0., 0., 1., 1., 0., 0., 1., 0., 0., 0., 0., 1., 1., 1., 0., 0.,
   1., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0.,
```

حال به ماتریس confusion توجه کنید

```
confusion_matrix(y_test, y_pred_default[:,1],labels = [1,0])
output: array([[33, 21],
   [12, 88]], dtype=int64)
```

حال آستانه دیگری را امتحان کنید تا کلاس ۱ شانس بیشتری برای انتخاب داشته باشد. ماتریس سردرگمی آن را مشاهده کنید

```
y_pred_low = binarize(y_pred_proba, threshold=0.2)
confusion_matrix(y_test, y_pred_low[:,1],labels=[1,0])
output: array([[47, 7],
   [42, 58]], dtype=int64)
```

با تغییر آستانه، احتمال پیش‌بینی کلاس ۱ افزایش یافت و اعداد ستون اول افزایش یافتند به طوری‌که اکنون مقدار ستون اول برابر ۸۹ = ۴۲ + ۴۷ می‌باشد.

پیش از این، مقدار ستون اول ۴۴ = ۱۲ + ۳۲ بود که عدد بسیار کمتری بود.

وقتی الگوریتم صفر را پیش‌بینی کند، احتمالاً درست است. وقتی یک را پیش‌بینی می‌کند، یعنی اشتباهی پیش‌بینی شده است. فرض کنید شما یک بیمارستان را اداره می‌کنید. به ندرت شخصی را به خانه می‌فرستید که معتقد هستید دیابت ندارد هرچند که دیابت دارد. هر زمان افرادی را که به دیابت مبتلا هستند به خانه بفرستید، نمی‌توان آن‌ها را زودتر معالجه کرد و هزینه‌های بیشتری را برای بیمارستان، شرکت‌های بیمه و خودشان متحمل می‌شوید.

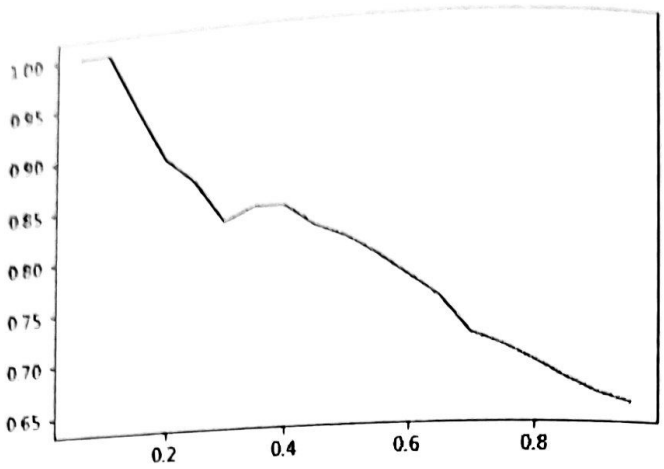
می توانید دقت آزمون را هنگام صفر اندازه‌گیری کنید. ستون دوم ماتریس سردرگمی را مشاهده کنید، [۵۸، ۷]. در این وضعیت، یا تقریباً ۵۸/(۵۸+۷) دقیق است. این مقدار پیش‌بینی منفی (NPV) نامیده می‌شود. برای محاسبه می‌توانید یک تابع بنویسیم.

جدول زیر روش‌های محاسبه specificity، sensitivity، PPV و NPV را نشان می‌دهد.

جدول (۵-۵): فرمول جدول سردرگمی

		پیش‌بینی		
		Positive Class	Negative Class	
واقعیت	Positive Class	True Positive (TP)	False Negative (FN)	Sensitivity = $TPR = \frac{TP}{TP + FN}$
	Negative Class	False Positive (FP)	True Negative (TN)	Specificity = $\frac{TN}{FP + TN}$
		PPV = $\frac{TP}{TP + FP}$	NPV = $\frac{TN}{FN + TN}$	

```
from __future__ import division #In Python 2.x
import matplotlib.pyplot as plt
def npv_func(th):
 y_pred_low = binarize(y_pred_proba, threshold=th)
 second_column = confusion_matrix(y_test,y_pred_low[:,1],labels=[1,0])[:,1]
 npv = second_column[1]/second_column.sum()
 return npv
npv_func(0.2)
output: 0.8923076923076924
ths = np.arange(0,1,0.05)
npvs = []
for th in np.arange(0,1.00,0.05):
 npvs.append(npv_func(th))
plt.plot(ths,npvs)
```



شکل (۵-۵): نمودار NPV

Receiver operating characteristic – ROC analysis

در همان خط بررسی NPV، معیارهای استانداردی وجود دارد که مقادیر خانه‌ها را در ماتریس سردرگمی بررسی می‌کند.

Sensitivity

sensitivity، مانند NPV در بخش قبلی، یک عملکرد ریاضی از سلول‌های ماتریس سردرگمی است. حساسیت، نسبت افرادی است که در این آزمایش با یک بیماری شرکت کرده اند و به درستی برچسب داشتن این بیماری، دیابت را دارند.

$$\text{Sensitivity} = \frac{People\ correctly\ labelled\ having\ diabets}{All\ People\ who\ have\ diabets}$$

$$\text{Sensitivity} = TPR = \frac{TP}{TP + FN}$$

نام دیگر آن true positive rate(TPR) نامیده می‌شود.

False Positive Rate (FPR)

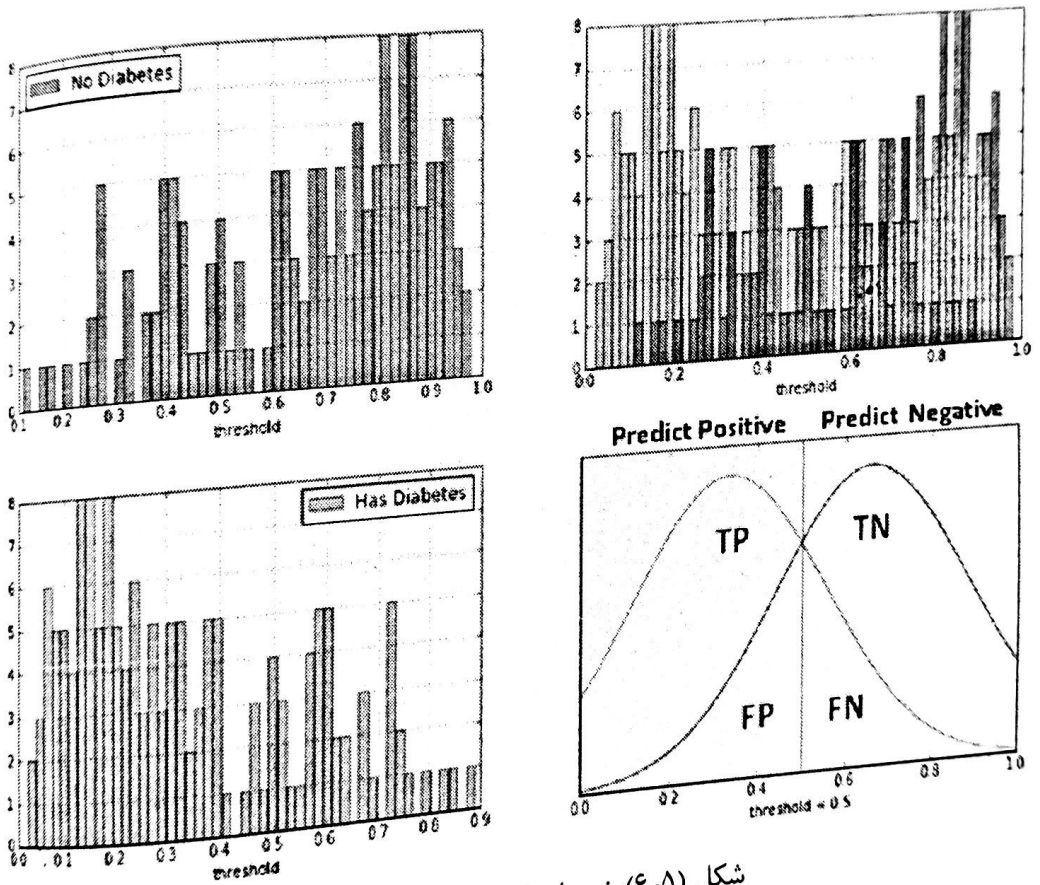
FPR میزان هشدارهای دروغین را توصیف می‌کند. تعداد افرادی که به اشتباه در بخش مبتلایان به دیابت قرار می‌گیرند اگرچه این طور نیست.

$$FPR = \frac{People\ incorrectly\ labelled\ having\ diabets}{People\ who\ do\ not\ have\ diabets}$$

$$FPR = \frac{FP}{FP + TN}$$

یک دیدگاه بصری

ما هم کلاس مثبت مبتلا به دیابت و هم گروه منفی فاقد دیابت را با هیستوگرام ترسیم می‌کنیم (در ستون سمت چپ در نمودار بعدی). هر کلاس تقریباً شبیه یک توزیع نرمال است. با SciPy، می‌توان بهترین توزیع‌های نرمال را پیدا کنیم. در پایین سمت راست توجه داشته باشید که آستانه روی ۰/۵ تنظیم شده است، تنظیمات پیش فرض در رگرسیون لجستیک می‌باشند. همان‌طور که مشاهده می‌شود آستانه باعث می‌شود منفی کاذب (FN) و مثبت کاذب (FP) را به طور ناقص انتخاب کنیم.



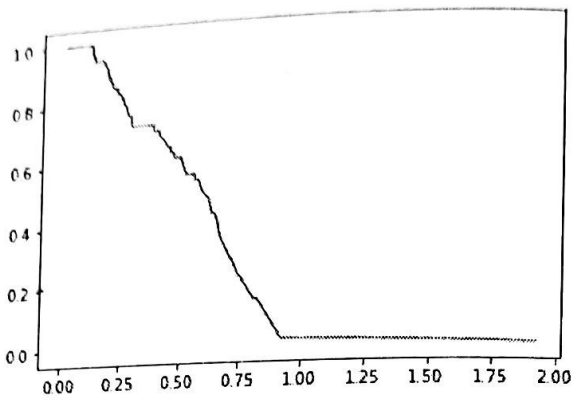
شکل (۵-۶): نمودارهای ماتریس سردرگمی

محاسبه TPR

scikit-learn توابع مناسبی برای محاسبه حساسیت یا TPR برای لجستیک رگرسیون با توجه به بردار احتمالات از کلاس مثبت دارد

- The TPR array (denoted by tpr)
- The FPR array (denoted by fpr)
- با آستانه مشخص(TPR and FPR (denoted by ths

```
from sklearn.metrics import roc_curve
fpr, tpr, ths = roc_curve(y_test, y_pred_proba[:,1])
plt.plot(ths,tpr)
```



شکل (۵-۷): TPR

بنابراین، هرچه آستانه پایین‌تر باشد، حساسیت نیز بهتر خواهد بود.

```
y_pred_th= binarize(y_pred_proba, threshold=0.1)
confusion_matrix(y_test, y_pred_th[:,1],labels=[1,0])
output: array([[54, 0],
   [67, 33]], dtype=int64)
y_pred_th = binarize(y_pred_proba, threshold=0.146)
confusion_matrix(y_test, y_pred_th[:,1],labels=[1,0])
output: array([[51, 3],
   [49, 51]], dtype=int64)
```

ماتریس سردرگمی در زمینه غیر پزشکی

با فرض اینکه شما یک بانکدار هستید و می‌خواهید تعیین کنید که مشتری مستحق وام رهنی برای خرید خانه است. در مرحله بعدی یک ماتریس سردرگمی احتمالی ایجاد می‌کنید که نشان می‌دهد آیا بر اساس داده‌های مشتری که در دسترس بانک است، به یک شخص وام رهنی تعلق می‌گیرد یا نه. در این زمینه، می‌توان اعداد را به هر سناریو اختصاص داد. وقتی هر سلول در ماتریس سردرگمی هزینه مشخصی دارد، یافتن بهترین طبقه‌بندی آسان‌تر است.

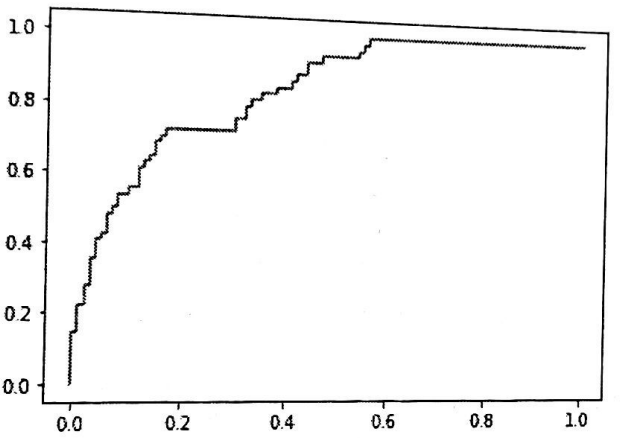
جدول (۵-۶): مثالی از جدول سردرگمی

		پیش‌بینی	
		وام ندادن	وام دادن
واقعیت	می تواند وام بگیرد	وام باید داده می‌شده است	وام کامل پرداخت می‌شود
	نمی تواند وام بگیرد	وام نباید داده شود	وام نباید داده می‌شده است

رسم منحنی ROC

منحنی ROC ابزاری تشخیصی برای دسته‌بندی بدون زمینه است. هیچ زمینه‌ای به این معنی است که ما هنوز نمی‌دانیم کدام نوع خطا (FP یا FN) مطلوب ماست. ROC نموداری از FPR (هشدارهای دروغین) در محور x و TPR (یافتن همه افراد با شرایطی که واقعاً آن را دارند) در محور y است. بدون زمینه، مشخص نیست که کدام خطا کمتر مطلوب ما می‌باشد.

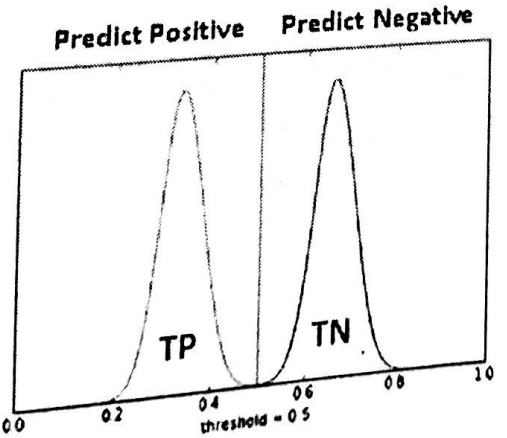
```
from sklearn.metrics import roc_curve
fpr, tpr, ths = roc_curve(y_test, y_pred_proba[:,1])
plt.plot(fpr,tpr)
```



شکل (۵-۸): نمودار TPR بر حسب FPR

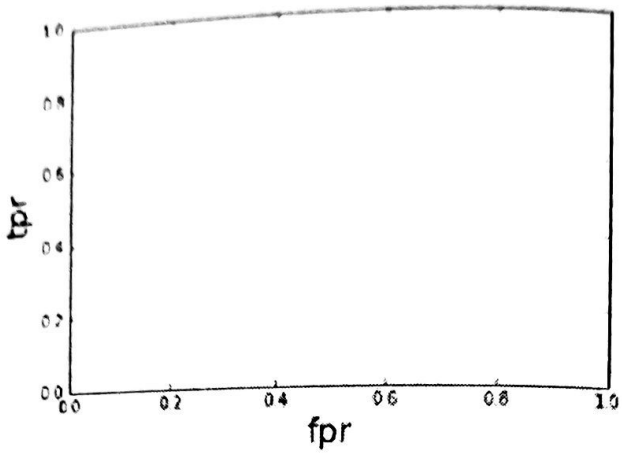
کلاسه‌بند کامل

یک طبقه‌بندی کننده کامل بدون در نظر گرفتن میزان هشدار کاذب (FAR) دارای TPR برابر ۱ است.



شکل (۵-۹): TN،TP

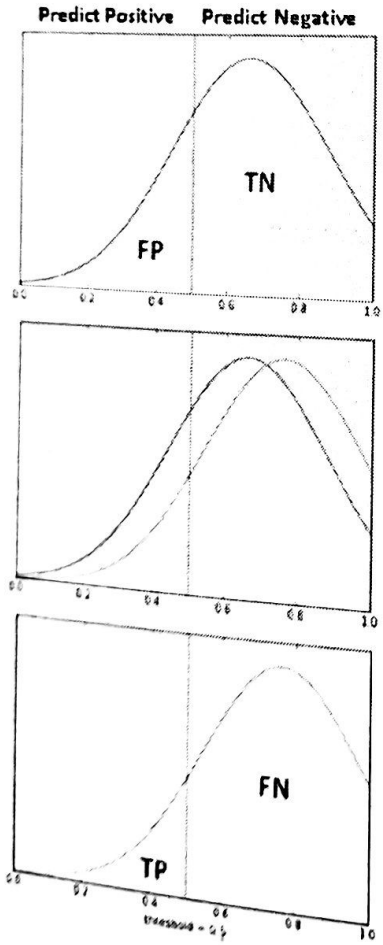
در نمودار قبل، FN بسیار کوچک است. بنابراین TPR نزدیک به ۱ است. منحنی ROC ان دارای شکل ⌐ است.



شکل (۵-۱۰): TPR بر حسب FPR

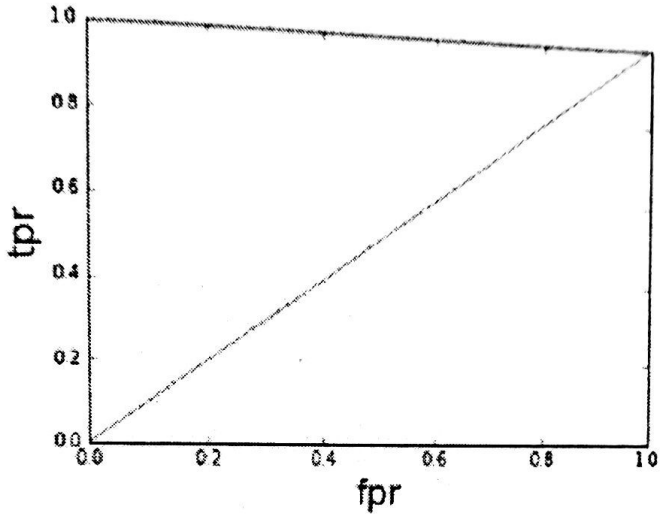
کلاسه‌بند ناقص

در تصاویر زیر، توزیع‌ها با هم همپوشانی دارند و نمی‌توان دسته‌ها را از یکدیگر تشخیص داد



شکل (۵-۱۱): همپوشانی در توزیع داده‌ها

در کلاسه‌بندی ناقص، FN و TN تقریباً برابر و FP و TP برابر هستند. بنابراین، با جابه‌جایی، نرخ مثبت TPR تقریباً برابر با نرخ منفی کاذب (FNR) FP / (FP + TN) است. حتی اگر آستانه را تغییر دهید، باز این دو مقدار یکسان هستند در نتیجه، منحنی ROC بدست می‌آوریم که یک خط مستقیم با شیب حدود ۱ است.



شکل (۵-۱۲): TPR بر حسب FPR

AUC - منطقه زیر منحنی ROC

مساحت کلاسه بند L شکل ۱ = ۱ × ۱ است. مساحت کلاسه بندی ناقص ۰.۵ است. برای اندازه‌گیری عملکرد کلاسه بند، scikit دارای یک تابع محاسبه منحنی AUC است.

```
from sklearn.metrics import auc
auc(fpr,tpr)
output: 0.8464814814814814
```

قرار دادن همه اینها کنار هم - مجموعه داده‌های سرطان پستان UCI

این مجموعه داده به لطف مخزن Street، N (1990)، UCI machine learning (https://archive.ics.uci.edu/ml/machine-learning-databases/breast-cancerwisconsin /breast-cancer-wisconsin.data) ارائه شده است، مدیسون، WI: دانشگاه ویسکانسین، گروه علوم کامپیوتر. پس از خواندن اطلاعات استناد/ مجوز، مجموعه داده را از UCI بارگیری می کنیم.

```
import numpy as np
import pandas as pd
```

```
data_web_address = data_web_address
="https://archive.ics.uci.edu/ml/machine-learning-databases/breast-cancer-
wisconsin/breast-cancer-wisconsin.data"
column_names =
['radius','texture','perimeter','area','smoothness','compactness','concavity','concav
e points','symmetry','malignant']
feature_names = column_names[:-1]
all_data = pd.read_csv(data_web_address , names=column_names)
```

اکنون، با خواندن مستندات، متغیر هدف روی ۲ (نداشتن سرطان) و ۴ (مبتلا به سرطان) تنظیم شده است. متغیرها را به صفر برای نداشتن سرطان و ۱ را برای داشتن سرطان تغییر دهید

```
#changing the state of having cancer to 1, not having cancer to 0
all_data['malignant'] = all_data['malignant'].astype(np.int)
all_data['malignant'] = np.where(all_data['malignant'] == 4, 1,0)
#4, and now 1 means malignant
all_data['malignant'].value_counts()
output: 0  458
1  241
Name: malignant, dtype: int64
```

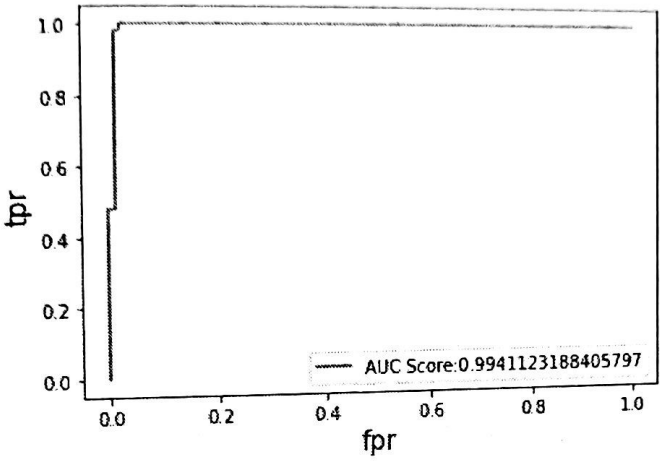
متغیرهای x و y را تعریف و سپس آن‌ها را به مجموعه‌های آموزشی و آزمایشی تقسیم کنید

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=7,stratify=y)
#Train and test the Logistic Regression. Use the method
#predict_proba().
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression()
lr.fit(X_train,y_train)
y_pred_proba = lr.predict_proba(X_test)
```

حال منحنی ROC را رسم و مقدار AUC را محاسبه می‌کنیم:

```
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc, roc_auc_score
fpr, tpr, ths = roc_curve(y_test, y_pred_proba[:,1])
auc_score = auc(fpr,tpr)
plt.plot(fpr,tpr,label="AUC Score:" + str(auc_score))
plt.xlabel('fpr',fontsize='15')
```

```
plt.ylabel('tpr',fontsize='15')
plt.legend(loc='best')
print(auc_score)
output: 0.9941123188405797
```



شکل (۵-۱۳): TPR بر حسب FPR

این کلاسه‌بند نسبتاً خوب عمل می‌کند.